

KVMSR+UDWeave: Extreme-Scaling with Fine-grained Parallelism on the UpDown Graph Supercomputer

Alexander Fell, Yuqing Wang, Tianshuo Su, Marziyeh Nourian, Wenyi Wang, Jose M. Monsalve-Diaz, Andronicus Rajasukumar, Jiya Su, Ruiqi Xu, Rajat Khandelwal, Tianchi Zhang, David F. Gleich, Yanjing Li, Hank Hoffmann, Andrew A. Chien

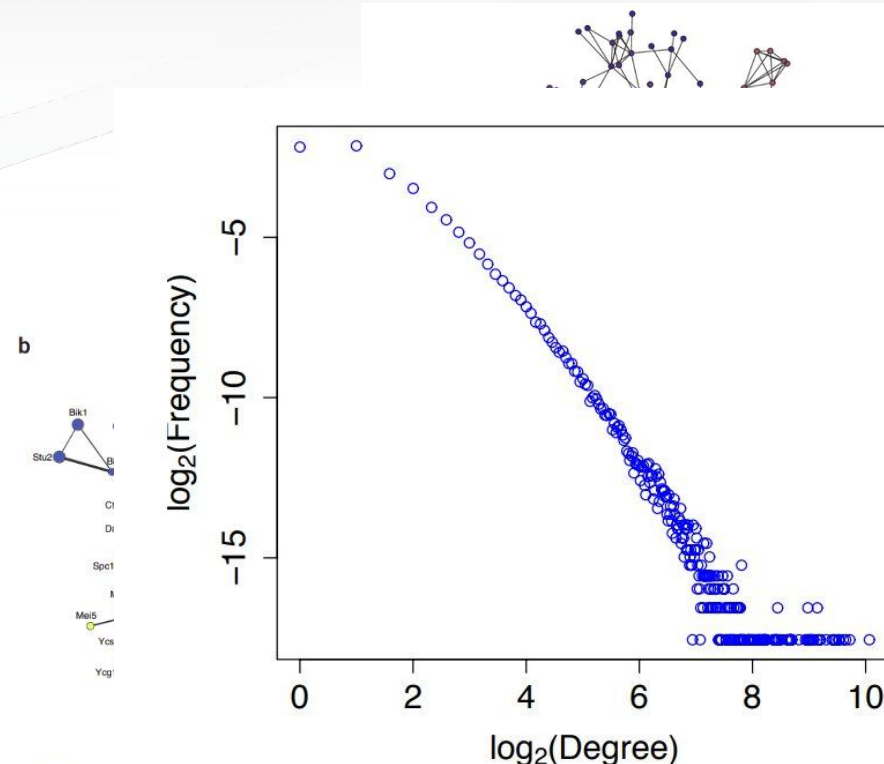
University of Chicago, Argonne National Labs, Purdue University

November 16, 2025



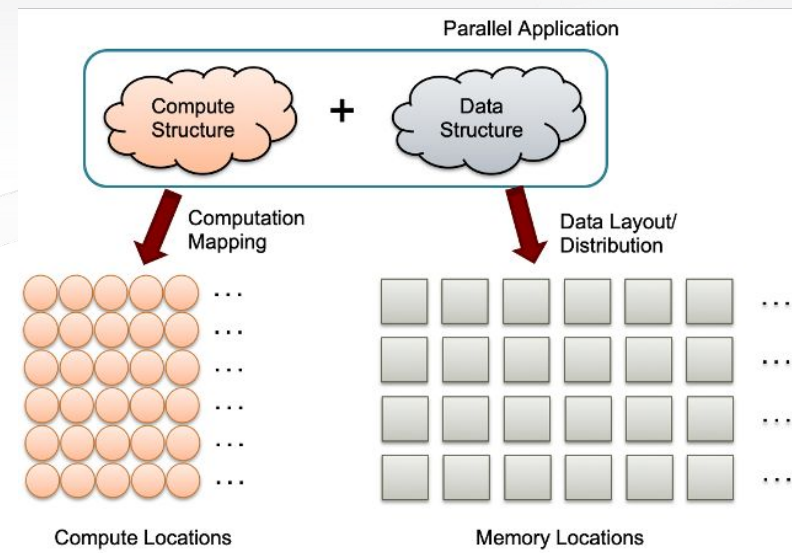
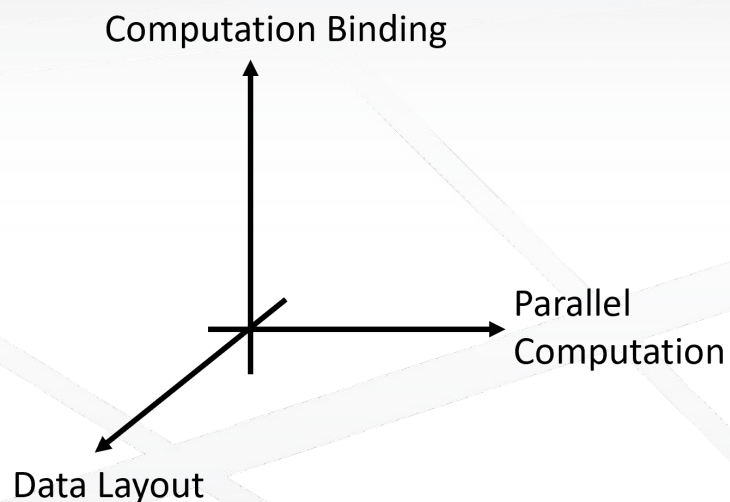
Massive-scale Graph Analytic Applications are Challenging

- Real-time Graph Processing poses extreme performance demands
 - Petabytes of Graph data
 - Billions of fine-grained tasks (vertex/edge)
 - Limited data reuse
- Skewed power-law degree distribution: extreme irregularity in data and compute
 - Unbalanced compute & poor data locality





Separating the 3 fundamental Dimensions of a Parallel Program



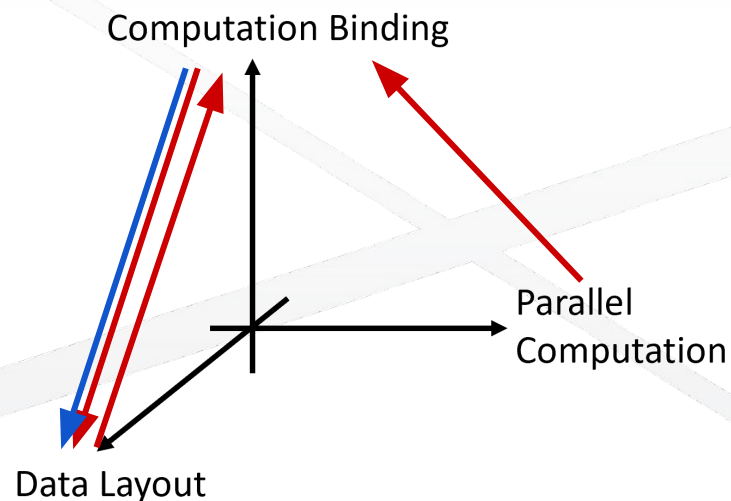
- Express and manage the parallelism
- Bind computation to parallel compute elements
- Layout data

All 3 must be solved to achieve high performance.

Our system enables programmers to optimize all three independently!



Existing Parallel Programming Models tie Dimensions Together



MPI:

Computation Binding $\leftrightarrow$ Data Placement

Parallel Computation $\leftrightarrow$ Computation Binding

PGAS:

Computation Binding $\leftrightarrow$ Data Placement

For irregular applications: coupling makes load balance and data distribution difficult.

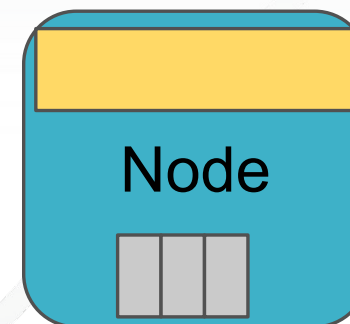
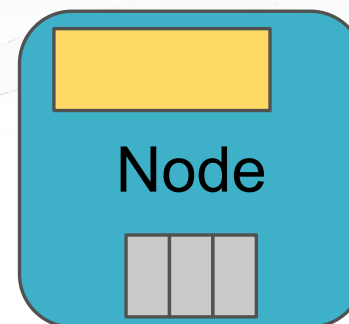
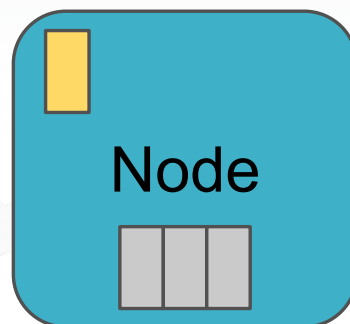
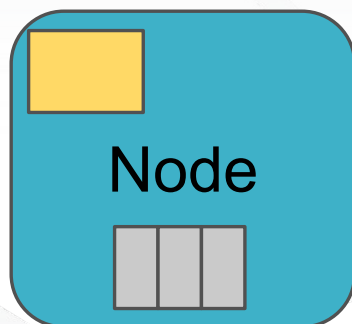




Example: MPI

Computation

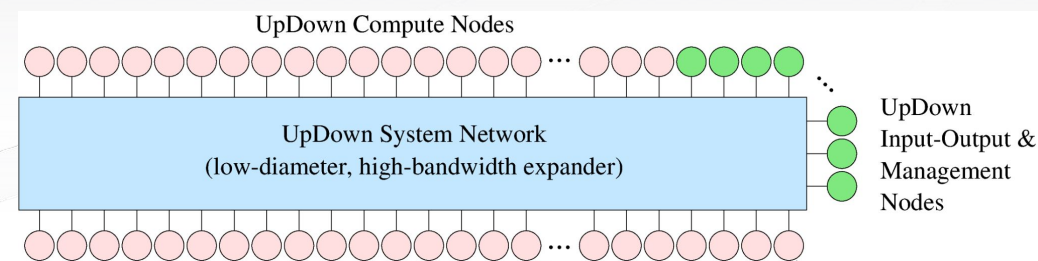
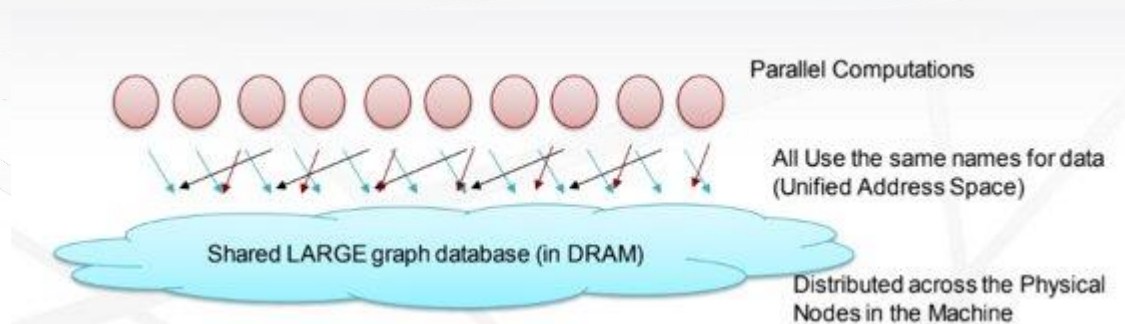
Data



- Regular Data Distribution => Work distribution
- Result: poor load balance



Context: UpDown Graph Supercomputer



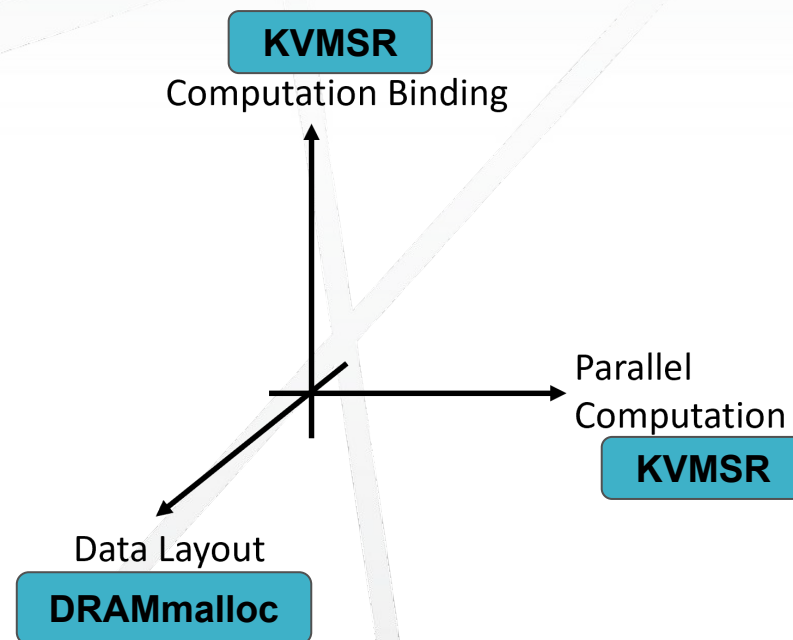
- UpDown System: massive fine-grained parallelism
 - Millions of multi-threaded processing units (Lanes)
 - Global address space
 - Petabytes of memory
 - Event-driven Tasks
 - Short threads with 1 cycle activation, switching, and termination
 - Fast and direct messaging between Lanes

2048
Lanes/Node



Addressing the 3 Dimensions

- Parallel Computation
 - **KVMSR**: Simple interface for describing and managing parallelism, Key-based approach
- Computation binding
 - **KVMSR**: Placement of Map and Reduce Tasks
 - **KVMSR**: Dynamic Load Balance w/ special machine features
- Data Placement/Layout
 - **DRAMmalloc**: Block/cyclic layout of global address space



Address the challenges of irregular computation at scale

- Skewed data, unbalanced load, fine-grained parallelism etc.



Parallel Computation

KVMSR:

```
kv_map()  
kv_reduce()
```

- User-defined key space
 - Supports different granularities
- Computation in map and reduce functions
 - Parallel computation across each key-value pair

```
event kv_map(Key key, uint64_t value0, uint64_t value1, double *value3) {  
    // source for the mapping function (parallel)  
    kv_map_emit(intermediate_key, intermediate_value);  
}
```

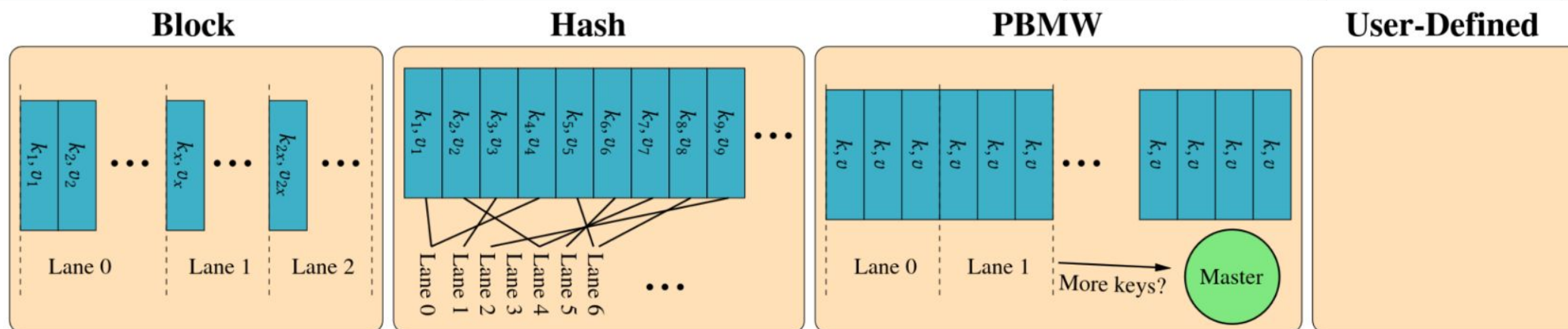
```
event kv_reduce(Key intermediate_key, uint64_t intermediate_value) {  
    // source for the reduce function (parallel)  
    kv_reduce_emit(output_key, output_value);  
}
```



Computation Binding

KVMSR:

`get_map_loc()`
`get_reduce_loc()`



```
uint32_t get_map_loc(Key key) {
    // compute the lane ID for the map function
    // for example:
    return (hash(key) % NRLanes) + 1stLane
}
```

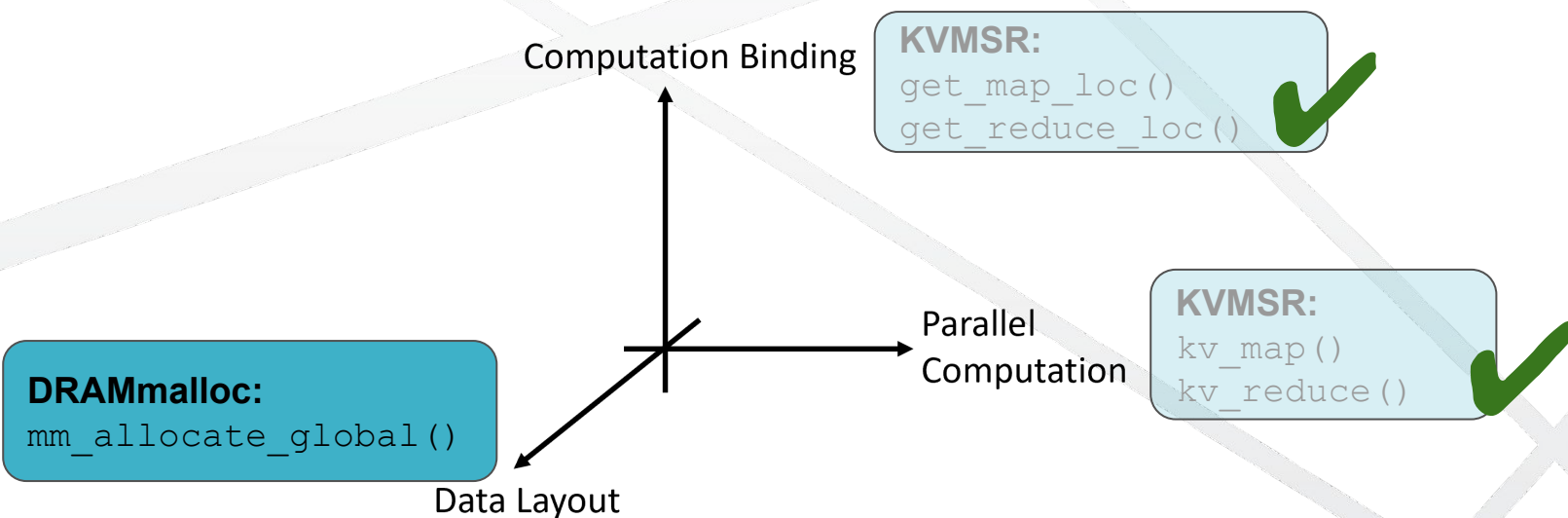


Data Layout

- DRAMmalloc: Controlling layout of global data structures
 - Block-cyclic layout
 - User controls: Block size, start node, #nodes

Examples of DRAMmalloc parameters:

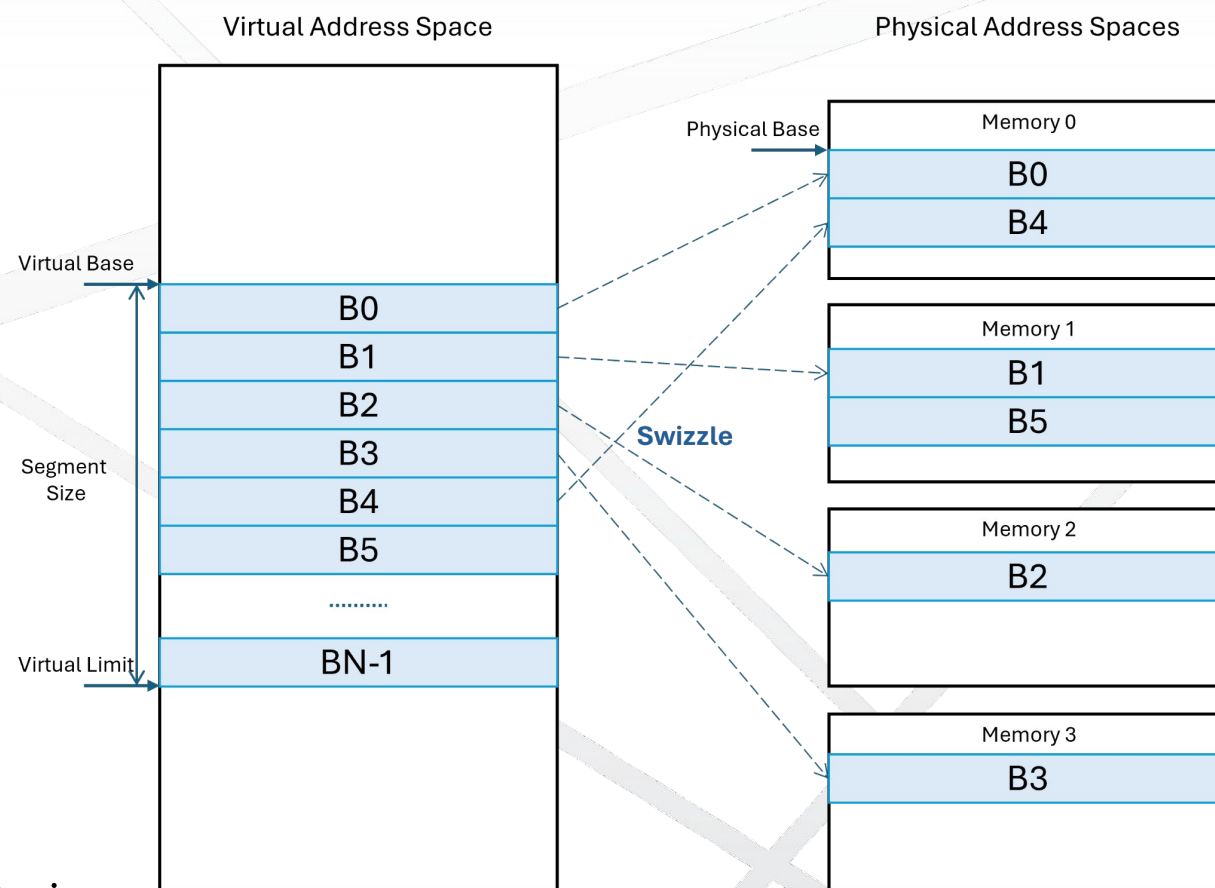
- (*, 0, 16k, 4096)
- (*, 0, 1k, 4096)
- (4TB, 4k, 8k, 1MB)





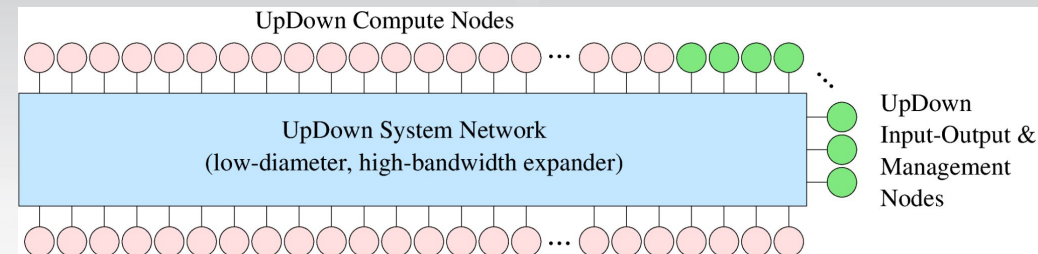
DRAMmalloc: Block Cyclic

```
void* DRAMmalloc(size, node1, num_nodes, block_size);
```





Evaluation

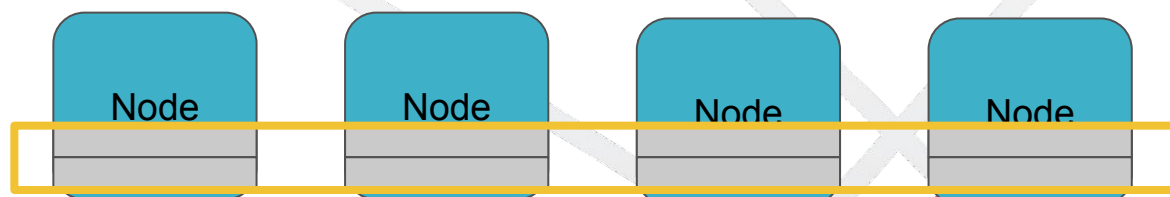
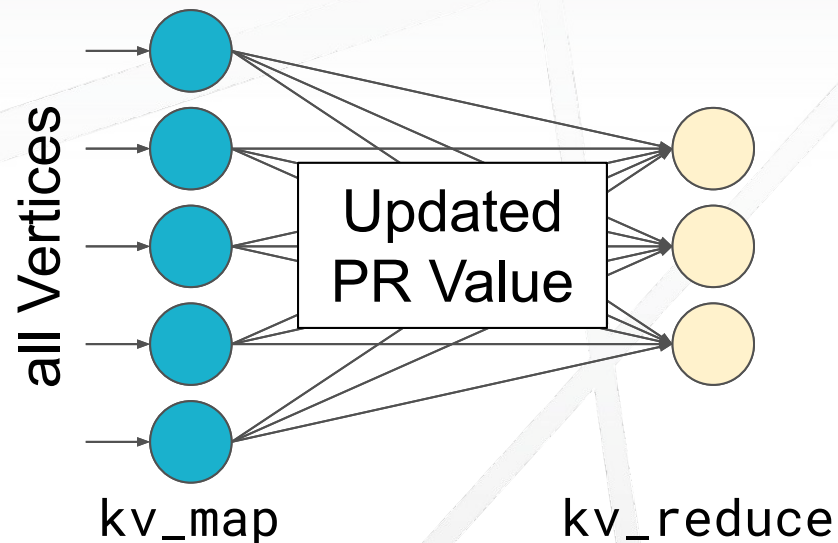


- UpDown System Configuration:
 - High Parallelism: 16k Nodes * 2048 lanes = 33M lanes
 - Efficient Fine-grained Task (10-100 insts)
 - High Memory bandwidth (10TB/s/node)
 - Global addressing of 8 PB of memory
- Kernels/Applications:
 - Graph algorithms: **PageRank**, BFS, TC
 - Novel streaming: Ingestion, Streaming pattern match
- Graphs
 - RMAT, ForestFire, Erdos-Renyi, some SNAP Graphs



PageRank

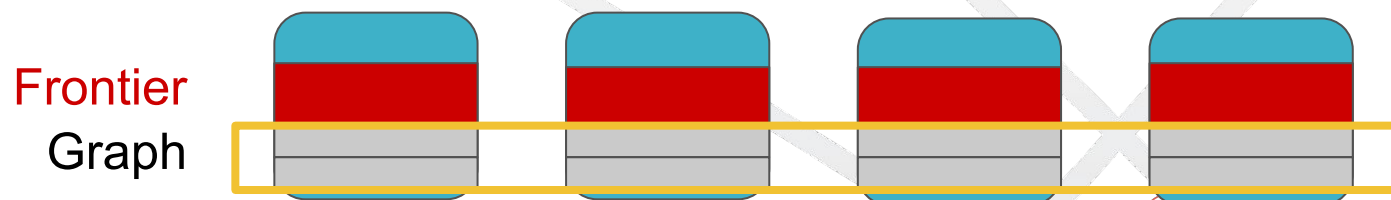
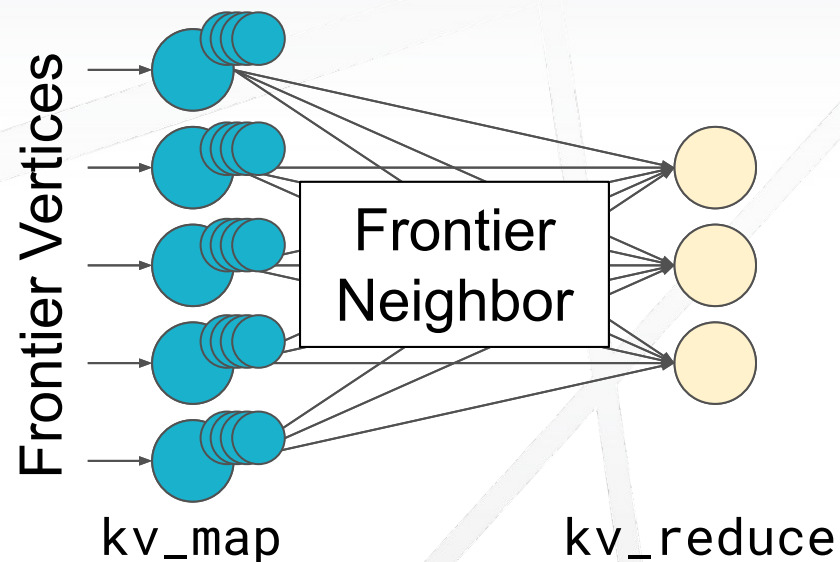
- Parallel Computation
 - kv_map: read vertex neighbors
 - kv_reduce: atomic PR value update
- Compute Binding
 - kv_map: block
 - kv_reduce: hash
- Data Layout
 - Graph: block-cyclic





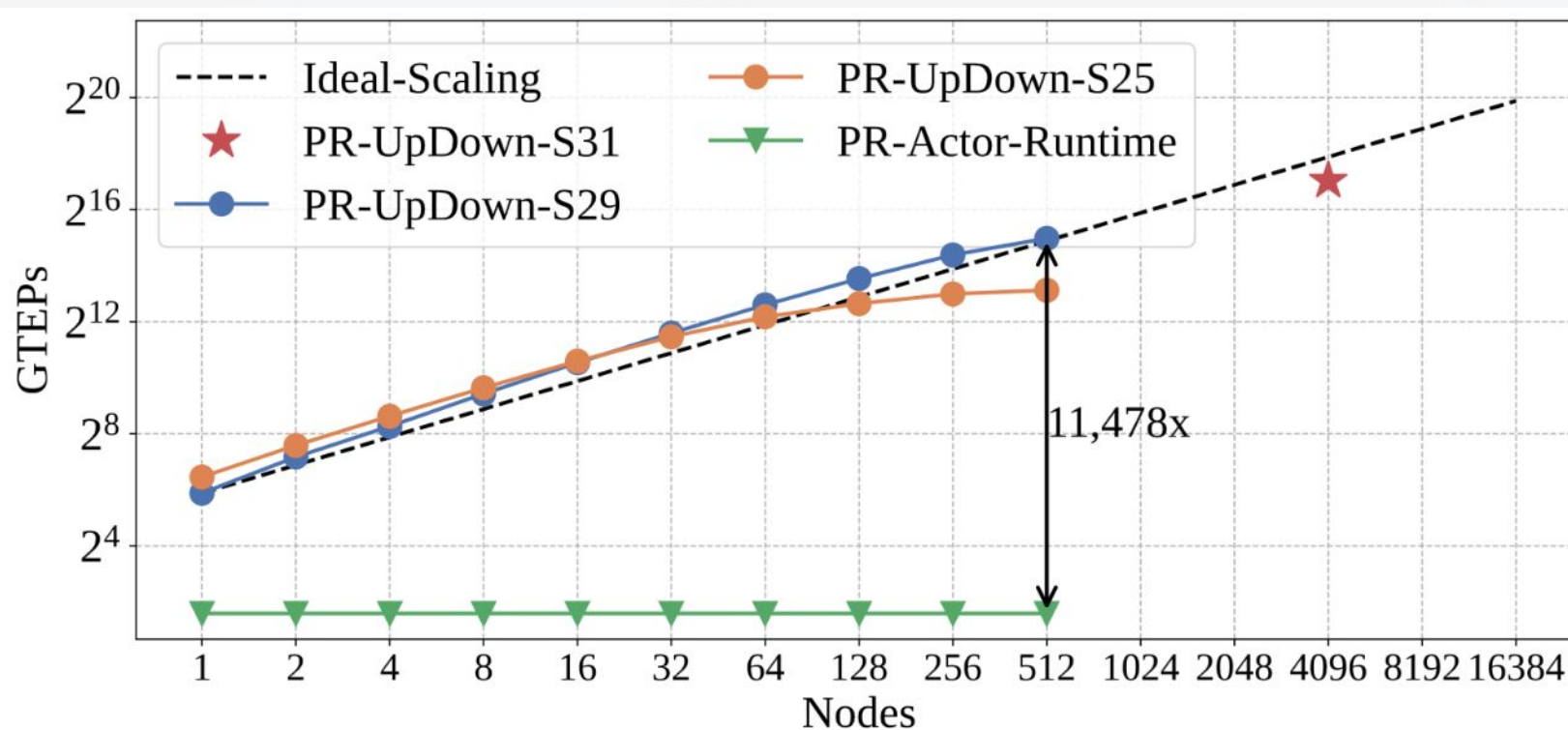
Breadth-First Search

- Parallel Computation
 - kv_map: compute BFS step on set of vertices
 - kv_reduce: store next frontier
- Compute Binding
 - kv_map: strided
 - kv_reduce: hash
- Data Layout
 - Frontier: Node-local (large block)
 - Graph: Block-cyclic





PageRank Results



In Comparison: Perlmutter 512 nodes: max. 11,478x improvement

Yuqing Wang, Charles Colley, Brian Wheatman, Jiya Su, David F. Gleich, and Andrew A. Chien. 2025. How Fast Can Graph Computations Go on Fine-grained Parallel Architectures. arXiv:2507.00949 [cs.DC] <https://arxiv.org/abs/2507.00949>



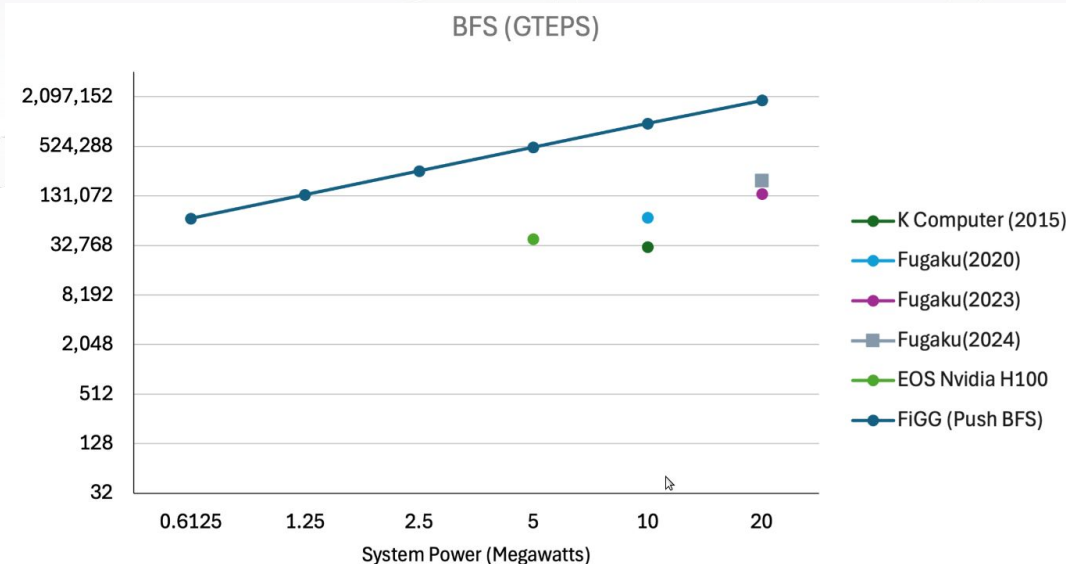
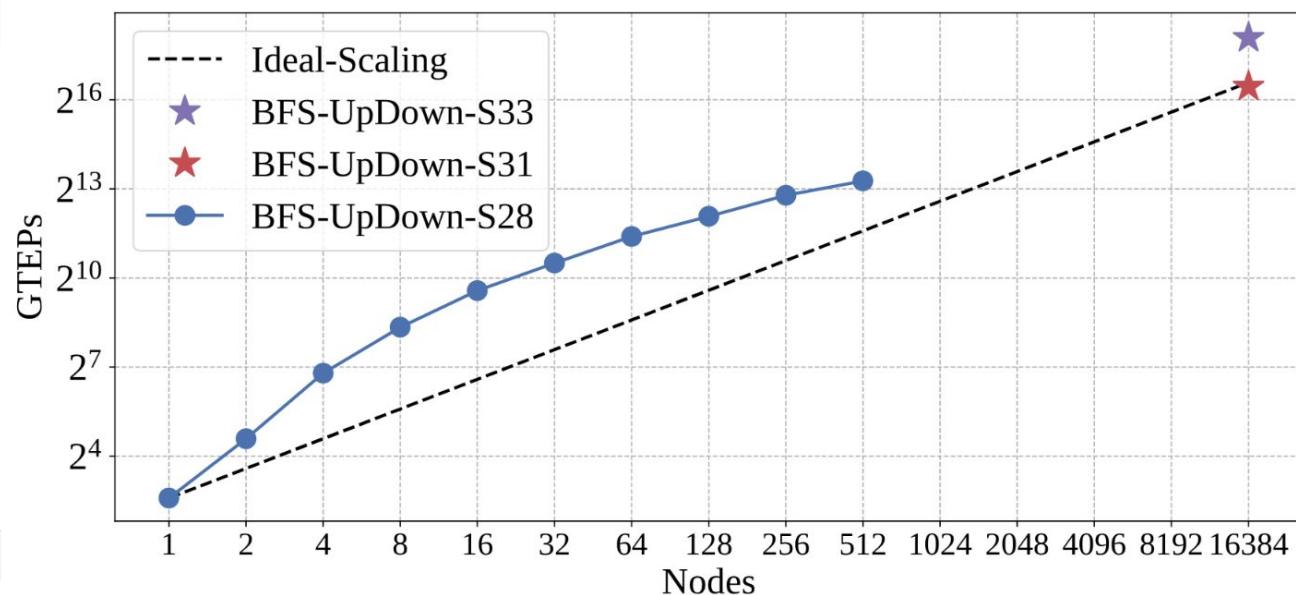
THE UNIVERSITY OF
CHICAGO

UpDown Project





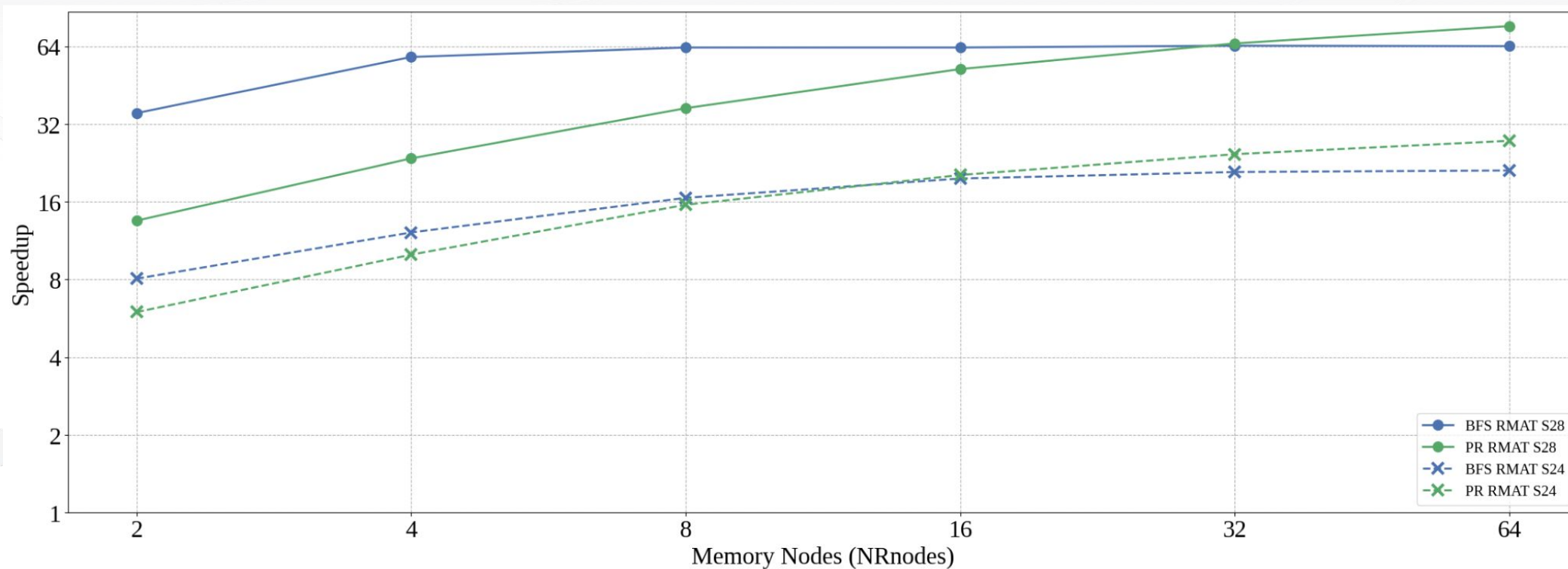
Breadth-First Search Results



Y. Wang et al. 2025. How Fast Can Graph Computations Go on Fine-grained Parallel Architectures. arXiv:2507.00949 [cs.DC]
<https://arxiv.org/abs/2507.00949>



Data Layout Impact

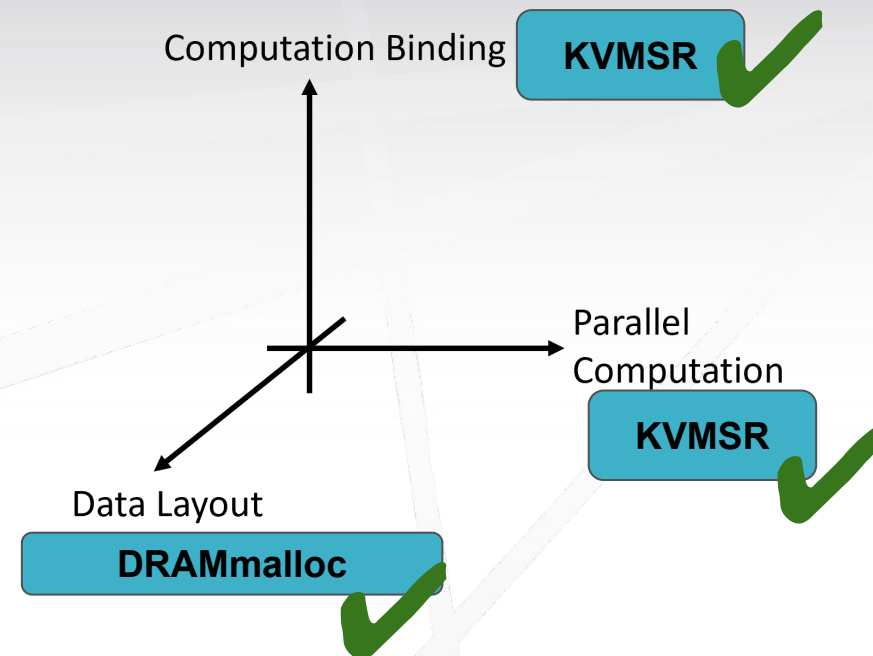


```
void* DRAMmalloc(size, node1, num_nodes, block_size);
```




Summary

- KVMSR+UDWeave+DRAMmalloc cleanly separates the 3 key dimensions for parallel programs
- **UDWeave**
 - System Programming language
- **KVMSR**
 - Computation Binding
 - Parallel Computation
- **DRAMmalloc**
 - Data Layout



On UpDown: breakthrough, scalable graph performance!



Questions?

More information:

<http://updown.cs.uchicago.edu/>

Y. Wang et al. "How Fast Can Graph Computations Go on Fine-grained Parallel Architectures," arXiv:2507.00949 [cs.DC] <https://arxiv.org/abs/2507.00949>, 2025
J. Su et al. "Scaling Triangle Counting and K-Truss on the UpDown Architecture," IEEE High Performance Extreme Computing Conference (HPEC), 2025
B. Wheatman et al. "Scalable Graph Algorithms on Distributed UpDown Accelerators," IEEE High Performance Extreme Computing Conference (HPEC), 2025
Y. Wang et al. "Updown: Combining scalable address translation with locality control," MEMO '24: International Workshop on Memory System, Management and Optimization, 2024
A. Rajasukumar et al. "UpDown: A Novel Architecture for Unlimited Memory Parallelism , in 10th International Symposium on Memory Systems (MEMSYS) 2024
A. Rajasukumar et al. "UpDown: Programmable fine-grained Events for Scalable Performance on Irregular Applications, " ARXIV, 2024

Thanks also to Jianru Ding, Brian Wheatman, Ziyi Zhang, Ahsan Pervaiz, Ryan Gillies, Michael Maire, Swann Perarnau